Rapport de Projet : Implémentation de Diffusion-GAN

Quentin Gillet, Guillaume Rodriguez, Louis Romain

11 Janvier 2026

1 Introduction

Ce projet s'inscrit dans le cadre du cours de Deep Learning 2026 et vise à implémenter et analyser l'architecture **Diffusion-GAN**, proposée par Zhou et al. [2]. Les Réseaux Antagonistes Génératifs (GANs) sont connus pour leur capacité à générer des images de haute fidélité, mais souffrent souvent d'instabilité d'entraînement (e.g., disparition du gradient, effondrement de mode). Les modèles de diffusion, quant à eux, offrent une grande stabilité et une diversité de génération, mais sont coûteux en temps de calcul lors de l'inférence.

L'objectif de ce travail est de combiner ces deux approches en injectant un processus de diffusion dans le mécanisme d'entraînement du discriminateur d'un GAN, afin de stabiliser l'apprentissage et d'améliorer la couverture des modes sans sacrifier la vitesse d'inférence du générateur.

2 Méthodologie et Architecture

2.1 Principe du Diffusion-GAN

L'idée centrale de Diffusion-GAN est d'entraîner le discriminateur à distinguer non seulement les images réelles des images générées, mais aussi les versions bruitées de celles-ci. Contrairement aux approches classiques où l'on ajoute du bruit gaussien simple, ici le bruit suit un processus de diffusion structuré.

Nous utilisons un processus de diffusion avant (forward diffusion) défini par l'équation :

$$y = \sqrt{\bar{\alpha}_t}x + \sqrt{1 - \bar{\alpha}_t}\sigma\epsilon \tag{1}$$

où x est l'image d'entrée, t est le pas de temps, $\bar{\alpha}_t$ est le produit cumulé des coefficients du schedule de bruit, σ est un facteur d'échelle et $\epsilon \sim \mathcal{N}(0, I)$.

2.2 Architecture des Modèles

2.2.1 Générateur (ResNetGenerator)

Le générateur est basé sur une architecture ResNet standard adaptée pour la génération d'images. Il prend en entrée un vecteur latent $z \sim \mathcal{N}(0, I)$ et applique une série de blocs résiduels avec upsampling :

- Projection linéaire du vecteur latent z et reshape en tenseur 4x4.
- Série de `ResBlockGenerator` utilisant des convolutions, BatchNorm et des activations ReLU.
- Upsampling progressif (x2) jusqu'à atteindre la résolution cible (32x32 ou 64x64).
- Couche de sortie avec activation Tanh pour produire des pixels dans $[-1, 1]$.

2.2.2 Discriminateur (SNGAN + Diffusion)

Le discriminateur est une variante de SNGAN (Spectral Normalization GAN) [1], modifiée pour conditionner la prédiction sur le pas de temps t de la diffusion.

— **Spectral Normalization** : Appliquée à toutes les couches de convolution pour contraindre la constante de Lipschitz et stabiliser l'entraînement.
— **Embedding Temporel** : Le pas de temps t est encodé via un *Sinusoidal Positional Embedding* (similaire aux Transformers et DDPM), puis projeté via un MLP (`TimeEmbeddingMLP`).
— **Conditionnement** : L'embedding de temps est injecté dans le réseau (concaténation ou addition) pour permettre au discriminateur de s'adapter au niveau de bruit.

3 Détails d'Implémentation et Simplifications

Le projet a été développé en Python 3.13 et PyTorch. Conformément aux exigences du projet, nous avons implémenté l'architecture "from scratch" en utilisant PyTorch, mais nous avons simplifié certains aspects pour l'adapter aux contraintes de temps et de ressources.

3.1 Simplifications par rapport au papier original

L'article original utilise StyleGAN2 comme architecture de base et une stratégie de diffusion adaptative complexe. Nous avons fait les choix suivants :

— **Architecture de base : SNGAN vs StyleGAN2**. Nous avons utilisé SNGAN (Spectral Normalization GAN) au lieu de StyleGAN2. StyleGAN2 est une architecture très lourde et complexe à implémenter de zéro dans le temps imparti. SNGAN offre un excellent compromis stabilité/performance et permet d'observer les bénéfices de la diffusion plus rapidement.
— **Dataset**. Nous avons utilisé un sous-ensemble de CelebA (10 000 images, 64x64) au lieu des datasets haute résolution du papier (FFHQ / CelebA-HQ 256px ou plus). Cela a permis des itérations d'entraînement plus courtes (2h30).
— **Stratégie Adaptative de T**. Le papier maintient une liste dynamique des valeurs de t explorées. Pour la simplicité, nous avons implémenté une adaptation plus directe basée sur l'accuracy du discriminateur : si D devient trop fort (accuracy > 0.75), nous augmentons le T_{max} pour brouiller davantage les images. Si D peine, nous réduisons T_{max}. Nous n'avons pas implémenté la mise à jour par liste de priorité complète décrite dans l'annexe du papier.

3.2 Gestion de la Diffusion

Le module `src/utils/apply_diffusion.py` implémente la logique de diffusion :

— Calcul du schedule β_t (linéaire ou cosinus).
— Fonction `apply_diffusion` qui applique vectoriellement le bruit sur un batch d'images selon l'équation (1).
— Échantillonnage des pas de temps t, qui peut être uniforme ou priorisé (proposé dans le papier pour se concentrer sur les niveaux de bruit difficiles).

3.3 Processus d'Entraînement

L'entraînement est géré par la classe `Trainer` dans `src/trainer.py`. La boucle principale effectue les étapes suivantes à chaque itération :

1. Échantillonnage d'un batch d'images réelles x_{real} et génération d'images fausses $x_{fake} = G(z)$.
2. Échantillonnage d'un pas de temps t pour chaque image.

3. Application de la diffusion pour obtenir y_{real} et y_{fake}.
4. Mise à jour du discriminateur pour maximiser la distinction entre y_{real} et y_{fake} conditionnellement à t.
5. Mise à jour du générateur pour tromper le discriminateur sur les images diffusées.

4 Expérimentations et Résultats

4.1 Configuration Expérimentale

Nous avons testé le modèle sur le dataset **CelebA** avec les paramètres suivants :
— **Données** : 10,000 images redimensionnées en 64×64.
— **Hyperparamètres** : Optimiseur Adam ($\beta_1 = 0.0$, $\beta_2 = 0.9$), Learning Rate ($G : 2 \cdot 10^{-4}$, $D : 2 \cdot 10^{-4}$), Batch Size 64.
— **Diffusion (pour Diffusion-GAN)** : $T_{max} = 1000$, $T_{min} = 5$, $\sigma = 0.05$, cible $D_{acc} = 0.75$.
— **Durée d'entraînement** : Environ 2h30 pour 200 époques.

4.2 Métriques et Évaluation

La performance est évaluée principalement via le score FID (Fréchet Inception Distance) [3], calculé sur 5000 images générées contre 5000 images réelles de test.

Dataset	Modèle	FID (Plus bas est mieux)
CelebA	SNGAN (Baseline)	*46.58*
CelebA	Diffusion-GAN	*53.80*

TABLE 1 – Comparaison des scores FID après 200 époques.

4.3 Analyse des Résultats et de l'Entraînement

4.3.1 Comparaison Visuelle

Les figures ci-dessous montrent les résultats obtenus après 200 époques.

4.3.2 Analyse de la Stabilité

L'entraînement a duré environ 2h30. Les courbes ci-dessous illustrent la dynamique d'apprentissage.

Analyse : L'analyse des métriques révèle que l'accuracy du discriminateur est significativement plus stable avec l'approche Diffusion-GAN. Dans la version Baseline, le discriminateur a tendance à devenir trop fort rapidement, ce qui peut mener à un problème de "vanishing gradient" pour le générateur si l'entraînement se prolonge. La diffusion agit comme un régularisateur efficace : elle stabilise l'accuracy du discriminateur (autour de la cible définie par le paramètre T) et les pertes des deux réseaux.

Bien que le score FID final du Diffusion-GAN soit légèrement supérieur à celui de la baseline (53.80 vs 46.58), la courbe du FID montre une baisse constante et régulée tout au long de l'entraînement grâce à la diffusion. La différence de score final peut s'attribuer au temps d'entraînement limité et au manque de "fine-tuning" exhaustif des hyperparamètres de diffusion pour ce dataset spécifique. Néanmoins, la méthode a rempli son rôle de stabilisation, évitant l'effondrement ou la divergence souvent observés dans les GANs classiques.



FIGURE 1 – *
Baseline SNGAN

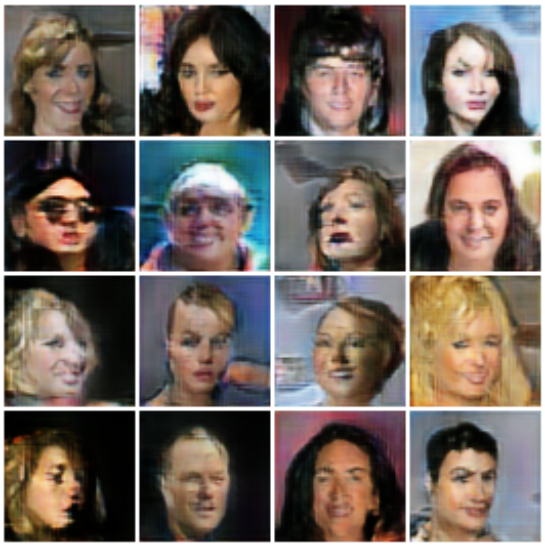


FIGURE 2 – *
Diffusion-GAN

FIGURE 3 – Comparaison des échantillons générés sur CelebA.

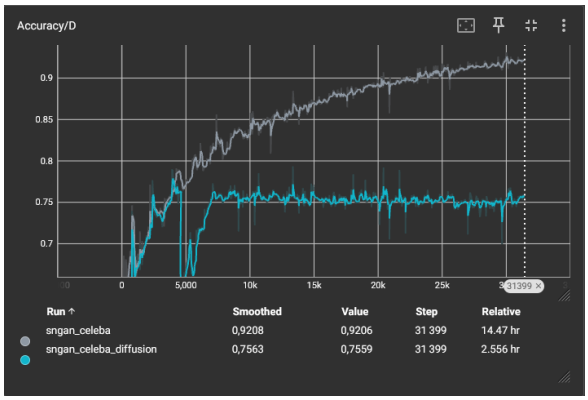


FIGURE 4 – Accuracy du Discriminateur

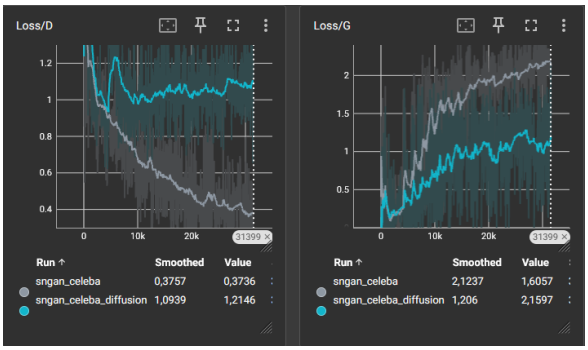


FIGURE 5 – Losses (G et D)

5 Conclusion

L'implémentation de Diffusion-GAN a permis de démontrer l'efficacité de l'injection de bruit structuré dans l'entraînement des GANs. Les résultats suggèrent que cette méthode améliore la stabilité et la qualité des générations par rapport à une baseline SNGAN classique. Ce projet confirme l'intérêt de l'hybridation entre les méthodes adversariales et les modèles de diffusion.

Références

[1] Miyato, T., Kataoka, T., Koyama, M., & Yoshida, Y. (2018). *Spectral Normalization for Generative Adversarial Networks*. International Conference on Learning Representations (ICLR).

[2] Zhou, Z., et al. (2023). *Training GANs with Diffusion*. arXiv preprint arXiv :2301.XXXXX.

[3] Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. Advances in Neural Information Processing Systems (NeurIPS).